# Chapter 1

# Score matching

The paper's networks are trained to approximate the score of the noised law, and its structured estimator is compared with them through the score. This chapter explains what "learning a score" means, derives the two objectives that make it possible without ever knowing the score (Hyvärinen's implicit and Vincent's denoising score matching), works out what each becomes under the Ornstein–Uhlenbeck channel of **??**, and then follows the code: the two output parameterisations, the three ways the divergence term is computed, the minibatch estimate of the loss, the estimand the paper reports, and the three information interfaces at which the comparison is run.

## 1.1 What learning a score means

A **score model** is a family of vector fields $s_\theta(\mathbf{x}, t)\colon \mathbb{R}^L \times (0,\infty) \to \mathbb{R}^L$ indexed by parameters $\theta$ (the weights of a network). The target is the score $S(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log P_t(\mathbf{x})$ of the noised law, and the natural measure of success is the **explicit score-matching** loss at one level,

$$J_{\mathrm{ESM}}(s) = \mathbb{E}_{P_t}\big|s(\mathbf{x}_t) - \nabla \log P_t(\mathbf{x}_t)\big|^2. \tag{1.1}$$

The difficulty is that $\nabla \log P_t$ is unknown: for real data one has samples of $P_t$ (or of $P_0$, which one can noise) and nothing else. In the paper's laboratory $S$ is known exactly (Section 2), which is what allows $J_{\mathrm{ESM}}$ to be measured; but the networks are still trained as they would be in practice, by one of the two surrogates below, which have the same minimiser as Equation (1.1) and can be estimated from samples. In both, $t$ is fixed for the moment; the paper's schedule enters in Section 1.6.

## 1.2 Hyvärinen's implicit score matching

**Theorem 1.1** (Implicit score matching)**.** *Let $p$ be a continuously differentiable density on $\mathbb{R}^L$ and let $s\colon \mathbb{R}^L \to \mathbb{R}^L$ be continuously differentiable, with $\mathbb{E}_p|s|^2 < \infty$, $\mathbb{E}_p|\nabla \log p|^2 < \infty$, $\mathbb{E}_p|\operatorname{div} s| < \infty$, and $p(\mathbf{x})\,|s(\mathbf{x})| \to 0$ as $|\mathbf{x}| \to \infty$. Then*

$$\mathbb{E}_p\big|s - \nabla \log p\big|^2 = 2\,\mathbb{E}_p\Big[\tfrac{1}{2}|s|^2 + \operatorname{div} s\Big] + \mathbb{E}_p|\nabla \log p|^2. \tag{1.2}$$

*The last term does not depend on $s$, so the **implicit score-matching** objective $J_{\mathrm{ISM}}(s) = \mathbb{E}_p[\frac{1}{2}|s|^2 + \operatorname{div} s]$ has the same minimiser as $J_{\mathrm{ESM}}$, and $J_{\mathrm{ISM}}$ involves only $s$ and samples of $p$.*

*Proof.* Scalar case first. Expand the square:

$$\int p\,(s - (\log p)')^2\,\mathrm{d}x = \int p\,s^2\,\mathrm{d}x - 2\int p\,s\,\frac{p'}{p}\,\mathrm{d}x + \int p\,((\log p)')^2\,\mathrm{d}x.$$

In the middle term $p \cdot p'/p = p'$, so it is $-2\int s\,p'\,\mathrm{d}x$, and integration by parts on $[-R, R]$ gives

$$\int_{-R}^{R} s\,p'\,\mathrm{d}x = [s\,p]_{-R}^{R} - \int_{-R}^{R} s'\,p\,\mathrm{d}x.$$

The boundary term tends to zero as $R \to \infty$ by the decay hypothesis, and the remaining integral converges by the integrability of $s'$ against $p$. Hence $\int p\,(s - (\log p)')^2 = \int p\,s^2 + 2\int p\,s' + \int p\,((\log p)')^2$, which is Equation (1.2) with $\operatorname{div} s = s'$. In $L$ dimensions the middle term is $-2\int \langle s, \nabla p\rangle\,\mathrm{d}\mathbf{x}$, and the divergence theorem on the ball $B_R$ gives $\int_{B_R}\langle s, \nabla p\rangle = \oint_{\partial B_R} p\,\langle s, \mathbf{n}\rangle\,\mathrm{d}\sigma - \int_{B_R} p \operatorname{div} s$; the surface term is bounded by the area of the sphere times $\sup_{|\mathbf{x}|=R} p|s|$, which tends to zero by hypothesis (the hypothesis is to be read as decay fast enough for this product to vanish). □

Two remarks. First, at the true score $J_{\mathrm{ISM}}(S) = -\frac{1}{2}\mathbb{E}|S|^2$, i.e. $\mathbb{E}_p[\operatorname{div}\nabla \log p] = -\mathbb{E}_p|\nabla \log p|^2$; combined with **??** this reproduces **??**. Second, the price of avoiding $\nabla \log p$ is the divergence, a sum of $L$ diagonal entries of the Jacobian of $s$, which is what Section 1.5 is about.

## 1.3 Vincent's denoising score matching

Suppose the samples of $p$ arise by corrupting clean samples: $\mathbf{a} \sim P_0$ and then $\mathbf{x} \sim p(\cdot \mid \mathbf{a})$, so that $p(\mathbf{x}) = \int P_0(\,\mathrm{d}\mathbf{a})\,p(\mathbf{x} \mid \mathbf{a})$. The conditional score $\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a})$ is known in closed form whenever the corruption is.

**Theorem 1.2** (Denoising score matching)**.** *Let $p(\mathbf{x} \mid \mathbf{a})$ be differentiable in $\mathbf{x}$ with $\mathbb{E}|\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a})|^2 < \infty$ under the joint law, and let $\mathbb{E}_p|s|^2 < \infty$. Then*

$$J_{\mathrm{DSM}}(s) := \mathbb{E}_{(\mathbf{a},\mathbf{x})}\big|s(\mathbf{x}) - \nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a})\big|^2 = J_{\mathrm{ESM}}(s) + \mathbb{E}_{(\mathbf{a},\mathbf{x})}\big|\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a}) - \nabla_{\mathbf{x}} \log p(\mathbf{x})\big|^2. \tag{1.3}$$

*The second term does not depend on $s$, so $J_{\mathrm{DSM}}$ and $J_{\mathrm{ESM}}$ have the same minimiser, and $J_{\mathrm{DSM}}$ needs only pairs $(\mathbf{a}, \mathbf{x})$.*

*Proof.* The key identity is that the marginal score is the conditional expectation of the conditional score,

$$\mathbb{E}\big[\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a}) \;\big|\; \mathbf{x}\big] = \nabla_{\mathbf{x}} \log p(\mathbf{x}), \tag{1.4}$$

because, with the posterior $p(\mathbf{a} \mid \mathbf{x}) = P_0(\mathbf{a})p(\mathbf{x} \mid \mathbf{a})/p(\mathbf{x})$,

$$\int p(\mathbf{a} \mid \mathbf{x})\,\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a})\,\mathrm{d}\mathbf{a} = \frac{1}{p(\mathbf{x})}\int P_0(\mathbf{a})\,\nabla_{\mathbf{x}} p(\mathbf{x} \mid \mathbf{a})\,\mathrm{d}\mathbf{a} = \frac{\nabla_{\mathbf{x}} p(\mathbf{x})}{p(\mathbf{x})}.$$

Now write $Y = \nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{a})$ and $S = \nabla \log p(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$. For any function $s$ of $\mathbf{x}$, the orthogonality argument of **??** applied to $Y$ gives $\mathbb{E}|s(\mathbf{x}) - Y|^2 = \mathbb{E}|s(\mathbf{x}) - S|^2 + \mathbb{E}|S - Y|^2$, which is Equation (1.3). □

The regression reading of the proof is worth stating on its own: $J_{\mathrm{DSM}}$ is a least-squares regression of the random target $Y$ on the input $\mathbf{x}$, and the least-squares regression function is the conditional expectation, which by Equation (1.4) is the score. The irreducible part of the loss is the conditional variance of the target.

**Corollary 1.3** (The channel)**.** *Under $\mathbf{x}_t = e^{-t}\mathbf{a} + \sqrt{\Delta_t}\mathbf{z}$, the conditional law is $\mathcal{N}(e^{-t}\mathbf{a}, \Delta_t \mathbf{I})$ and the target is*

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}_t \mid \mathbf{a}) = -\frac{\mathbf{x}_t - e^{-t}\mathbf{a}}{\Delta_t} = -\frac{\mathbf{z}}{\sqrt{\Delta_t}}. \tag{1.5}$$

*Identity Equation* (1.4) *is Tweedie's identity* **??** *in another guise, and the irreducible loss is $\mathbb{E}|-\mathbf{z}/\sqrt{\Delta_t} - S|^2 = (e^{-2t}/\Delta_t^2)\,\mathrm{MMSE}_t$ by* **??**.

The target's variance is $L/\Delta_t$ per example, which diverges as $t \to 0$; a network asked to output the score directly would have to produce values of size $1/\sqrt{\Delta_t}$ at the lowest levels. The parameterisations of the next section are the standard way to avoid that.

## 1.4 The two output parameterisations

Every network of the paper maps a noised sequence and a level to an output $f_\theta(\mathbf{x}, t) \in \mathbb{R}^L$, and the output is converted to a posterior mean and then to a score by **??**. The code supports two conventions; they are implemented by `Denoiser.posterior_mean`, `_score_from_out` and `dsm_loss` in `src/markov_score/nn_torch.py`.

**Definition 1.4** (`eps` and `x0`)**.** With $\alpha = e^{-t}$:

- `eps`: the output predicts the noise, target $\mathbf{z}$; the posterior mean is $\widehat{m} = (\mathbf{x} - \sqrt{\Delta_t}\, f)/\alpha$ and the score is $\widehat{s} = -(\mathbf{x} - \alpha\widehat{m})/\Delta_t = -f/\sqrt{\Delta_t}$.
- `x0`: the output predicts the clean sequence, target $\mathbf{a}$; the posterior mean is $\widehat{m} = f$ and the score is $\widehat{s} = -(\mathbf{x} - \alpha f)/\Delta_t$.

The training loss in both cases is the mean squared distance between output and target over the batch, $\mathbb{E}|f - \mathbf{z}|^2$ or $\mathbb{E}|f - \mathbf{a}|^2$.

**Proposition 1.5** (Both losses are weighted denoising score matching)**.** *With $s^* = -(\mathbf{x} - \alpha\mathbf{a})/\Delta_t$ the target of Equation* (1.5)*,*

$$|f - \mathbf{z}|^2 = \Delta_t\, |\widehat{s} - s^*|^2 \quad (\texttt{eps}), \qquad |f - \mathbf{a}|^2 = \frac{\Delta_t^2}{\alpha^2}\, |\widehat{s} - s^*|^2 \quad (\texttt{x0}).$$

*Hence the `eps` loss is $J_{\mathrm{DSM}}$ weighted by $\Delta_t$, and the `x0` loss is $J_{\mathrm{DSM}}$ weighted by $\Delta_t^2/\alpha^2 = \Delta_t/\mathrm{SNR}_t$. Both have the score as their minimiser at every fixed $t$.*

*Proof.* From $\widehat{s} = -(\mathbf{x} - \alpha\widehat{m})/\Delta_t$ and $s^* = -(\mathbf{x} - \alpha\mathbf{a})/\Delta_t$, $\widehat{s} - s^* = \alpha(\widehat{m} - \mathbf{a})/\Delta_t$, so $|\widehat{m} - \mathbf{a}|^2 = (\Delta_t^2/\alpha^2)|\widehat{s} - s^*|^2$; this is the `x0` case since $\widehat{m} = f$. In the `eps` case $\widehat{m} - \mathbf{a} = (\mathbf{x} - \sqrt{\Delta_t} f)/\alpha - (\mathbf{x} - \sqrt{\Delta_t}\mathbf{z})/\alpha = -\sqrt{\Delta_t}(f - \mathbf{z})/\alpha$, so $|f - \mathbf{z}|^2 = (\alpha^2/\Delta_t)|\widehat{m} - \mathbf{a}|^2 = \Delta_t|\widehat{s} - s^*|^2$. □

The weights matter once levels are mixed (Section 1.6): `eps` weights the levels almost equally in score units at high noise ($\Delta_t \approx 1$) and suppresses the lowest levels ($\Delta_t \approx 2t$); `x0` suppresses them more strongly. The paper trains both and lets the screen choose (Appendix C, "both output parameterisations"). The conversion $\widehat{s} = -f/\sqrt{\Delta_t}$ for `eps` is the reason a network output of order one corresponds to a score of order $1/\sqrt{\Delta_t}$, as the target requires.

## 1.5 Computing the divergence

Implicit score matching needs $\operatorname{div} \widehat{s} = \sum_i \partial \widehat{s}_i/\partial x_i = \operatorname{tr} J$ for every example in a batch, where $J = \partial\widehat{s}/\partial\mathbf{x}$ is the $L \times L$ Jacobian of the network's score. Automatic differentiation gives Jacobian-transpose-vector products cheaply: for any $\mathbf{v}$, one backward pass through $\langle \widehat{s}, \mathbf{v}\rangle$ returns $J^\top \mathbf{v}$, since $\partial_j \sum_i \widehat{s}_i v_i = \sum_i v_i J_{ij}$. The function `divergence` in `src/markov_score/nn_torch.py` offers three ways to turn such products into a trace.

### 1.5.1 The exact trace by $L$ backward passes

Taking $\mathbf{v} = \mathbf{e}_i$ returns the $i$-th row of $J$, whose $i$-th entry is $J_{ii}$. Summing over $i$ costs $L$ backward passes per batch (the `"exact"` branch: for each $i$, the gradient of $\sum_{\text{batch}} \widehat{s}_i$ with respect to $\mathbf{x}$, read at coordinate $i$). At $L = 32$ this is thirty-two passes for every training step, and it is what the frozen matched arm used for the window head (Appendix C: "computed exactly for the window head").

### 1.5.2 Hutchinson's estimator

**Proposition 1.6** (Hutchinson)**.** *Let* $\mathbf{v}$ *be a random vector with* $\mathbb{E}[\mathbf{v}\mathbf{v}^\top] = \mathbf{I}$*, independent of* $J$*. Then* $\mathbb{E}[\mathbf{v}^\top J\mathbf{v}] = \operatorname{tr} J$*. If* $\mathbf{v} \sim \mathcal{N}(0, \mathbf{I})$*,*

$$\operatorname{Var}(\mathbf{v}^\top J\mathbf{v}) = \|J\|_F^2 + \operatorname{tr}(J^2) = \|J\|_F^2 + \langle J, J^\top\rangle,$$

*which equals* $2\|J\|_F^2$ *when* $J$ *is symmetric.*

*Proof.* $\mathbb{E}[\mathbf{v}^\top J\mathbf{v}] = \sum_{ij} J_{ij}\mathbb{E}[v_i v_j] = \sum_i J_{ii}$. For the variance with Gaussian $\mathbf{v}$, use $\mathbb{E}[v_i v_j v_k v_l] = \delta_{ij}\delta_{kl} + \delta_{ik}\delta_{jl} + \delta_{il}\delta_{jk}$:

$$\mathbb{E}\big[(\mathbf{v}^\top J\mathbf{v})^2\big] = \sum_{ijkl} J_{ij}J_{kl}\mathbb{E}[v_i v_j v_k v_l] = (\operatorname{tr} J)^2 + \sum_{ij} J_{ij}^2 + \sum_{ij} J_{ij}J_{ji},$$

and subtract $(\operatorname{tr} J)^2$. □

The score of a density has a symmetric Jacobian (**??**); a network's need not, but is close to symmetric when it is close to a score. With one standard-normal probe per example per step (the `"hutchinson"` branch, used by the frozen matched arm for the transformer), the loss estimate is unbiased but its variance adds to the minibatch noise of Section 1.6. Hutchinson (1990) used Rademacher probes ($v_i = \pm 1$), for which the diagonal contributes no variance and $\operatorname{Var} = 2\sum_{i\neq j} J_{ij}J_{ji}$, at most the Gaussian value; the code uses Gaussian probes. For scale: if $J = \mathbf{I}_{32}$ a Gaussian probe estimates the trace 32 with standard deviation $\sqrt{2 \cdot 32} = 8$.

### 1.5.3 Coloured probes for a banded Jacobian

The window head of the paper computes its output at site $i$ from the inputs at sites $|j - i| \le r$ only (radius $r$; `receptive_radius` in the code). Its Jacobian is therefore **banded**: $J_{ij} = 0$ for $|i - j| > r$. For such a map the exact trace costs $2r + 1$ backward passes rather than $L$.

**Proposition 1.7** (Coloured probes)**.** *Let* $J_{ij} = 0$ *whenever* $|i - j| > r$*, put* $p = 2r + 1$*, and for* $k = 0, \dots, p - 1$ *let* $\mathbf{v}_k \in \{0, 1\}^L$ *have entries* $(\mathbf{v}_k)_j = \mathbf{1}\{j \equiv k \pmod p\}$*. Then* $\sum_{k=0}^{p-1} \mathbf{v}_k^\top J\mathbf{v}_k = \operatorname{tr} J$ *exactly.*

*Proof.* $\mathbf{v}_k^\top J\mathbf{v}_k = \sum_{i\equiv k}\sum_{j\equiv k} J_{ij}$. If $i \equiv j \pmod p$ and $i \neq j$ then $|i - j| \ge p = 2r + 1 > r$, so $J_{ij} = 0$. Only $i = j$ survives: $\mathbf{v}_k^\top J\mathbf{v}_k = \sum_{i\equiv k} J_{ii}$. Every $i$ has exactly one residue class, so the sum over $k$ is $\sum_i J_{ii}$. □

This is the colouring of a banded Jacobian of Curtis, Powell and Reid (1974): two columns may share a probe when no row has a nonzero in both, and the residue classes modulo $2r + 1$ are such a partition. The code (`"coloured"` branch) computes, for each $k$, one backward pass $J^\top \mathbf{v}_k$ and then the inner product with $\mathbf{v}_k$, exactly as for Hutchinson but with $p$ deterministic probes and no variance. Zero padding at the ends of the chain only removes dependencies, so the boundary sites are covered. The branch refuses a model that declares no receptive radius, since for a dense Jacobian the cross terms would not vanish. The identity is checked against the $L$-pass trace by the test `test_coloured_trace_equals_exact_trace_for_the_window_head` in `tests/test_nn_torch.py`. For the radius-2 head this is five passes in place of thirty-two.

**Example 1.8** (Seven sites, radius one)**.** With $L = 7$, $r = 1$ and $p = 3$ the probes are $\mathbf{v}_0 = \mathbf{e}_1 + \mathbf{e}_4 + \mathbf{e}_7$, $\mathbf{v}_1 = \mathbf{e}_2 + \mathbf{e}_5$ and $\mathbf{v}_2 = \mathbf{e}_3 + \mathbf{e}_6$. For a tridiagonal $J$, $\mathbf{v}_0^\top J\mathbf{v}_0 = J_{11} + J_{44} + J_{77} + (J_{14} + J_{41} + J_{47} + J_{74} + J_{17} + J_{71})$ and every bracketed entry is zero because its indices differ by at least three. Adding the three probes gives $J_{11} + \cdots + J_{77}$.

## 1.6 Why the loss is estimated per batch

The population objective is an expectation; training minimises it by stochastic gradient descent on unbiased minibatch estimates. Under the paper's recipe (`TrainConfig` and `train` in `src/markov_score/nn_torch.py`) one step draws a batch of 64 examples, and for each example a level $t$ uniformly from the twelve of **??** and a fresh corruption $\mathbf{z}$; the loss is the mean of the per-example losses; the gradient is taken; its norm is clipped at 1; and Adam takes a step at learning rate $10^{-3}$ or $3 \times 10^{-3}$. Because $t$ and $\mathbf{z}$ are drawn independently of $\theta$, the batch mean is an unbiased estimate of

$$\bar{J}(\theta) = \frac{1}{12} \sum_{k=1}^{12} w(t_k)\, J_{\mathrm{DSM}}^{(t_k)}(s_\theta(\cdot, t_k)) \tag{1.6}$$

with the weights $w$ of Proposition 1.5, and its gradient is an unbiased estimate of $\nabla \bar{J}$. The minimiser of Equation (1.6) over all vector fields is the score at every level simultaneously, since the levels do not compete (the field may depend on $t$); with a finite network they do compete, and the weights decide the compromise. Checkpoints are saved on a ladder up to 32,000 steps and one is selected afterwards on a validation bundle; the selection rule is part of the interface (Section 1.8). For implicit score matching the same loop draws 64 examples from the fixed set of noised views, computes $\frac{1}{2}|\widehat{s}|^2 + \operatorname{div} \widehat{s}$ per example with one of the three divergence branches, and averages; with a Hutchinson probe the estimate is still unbiased, since the probe is drawn independently of $\theta$, at the cost of extra variance.

## 1.7 The estimand: schedule-pooled relative score error

**Definition 1.9** (The estimand)**.** The paper's headline number for any estimator $\widehat{s}$ is the **schedule-pooled relative score error** (`experiments/protocol.py`, `estimand`; Appendix C, "Estimand"),

$$\operatorname{err}(\widehat{s}) = \left( \frac{\sum_t \sum_j \sum_i \big(\widehat{s}_t(\mathbf{x}_j)_i - s_t(\mathbf{x}_j)_i\big)^2}{\sum_t \sum_j \sum_i s_t(\mathbf{x}_j)_i^2} \right)^{1/2}, \tag{1.7}$$

where $t$ runs over the twelve levels, $j$ over the sequences of a test bundle noised at that level, $i$ over the sites of a region, and $s_t$ is the tree-exact reference score. Both sums are formed before the square root.

Three features of Equation (1.7) matter for reading the paper. It is *relative*: an estimator that outputs zero scores 1, and the isotropic score $-\mathbf{x}$ and the LMMSE twin give the two reference values of the paper's oracle ladder. It is *pooled* rather than averaged over levels: the numerator and the denominator are energies summed over the schedule, so a level contributes in proportion to its score energy, which by **??** is close to $L/\Delta_t$ per sequence at low noise. Taking $1/\Delta_t$ as a proxy for the per-site energy, the twelve levels of **??** carry the weights $10.5, 7.4, 5.3, 3.8, 2.8, 2.1, 1.65, 1.35, 1.16, 1.06, 1.02, 1.00$, whose sum is 39.1; the lowest four levels carry about 69% of the pooled denominator and the highest four about 11%. A pooled error is therefore a low-noise-dominated summary, which the paper says in words (Section 3, "What this says about score learning") and which explains why the fourth-order Edgeworth correction, exact at high noise, is worse than the twin when pooled (**??**). Finally it is computed on *regions*: all 32 sites (the headline), the bulk, sites 5 to 28 (four trimmed at each end, used for every selection so that boundary effects do not drive the choice of checkpoint), and the centre site (a diagnostic). Ratios between arms are formed within seed and averaged, and intervals are percentile bootstraps over seeds (Appendix C).

## 1.8 The three information interfaces

The paper's Section 5 declares three interfaces and reports the structured estimator against the networks at each. The definitions below follow `experiments/e03_baselines.py` line by line.

**Definition 1.10** (Interface A: the standard denoising recipe)**.** The network sees the $M$ clean training sequences. At every step it draws a batch, a level per example from the twelve, and a fresh corruption per example, and minimises the `eps` or `x0` denoising loss (`_train_select` with `loss="dsm"`; `_draw_batch` in `nn_torch.py`). The checkpoint is selected on a validation bundle of 1024 sequences, scored by Equation (1.7) on the bulk sites against the tree-exact reference under the true kernel.

**Definition 1.11** (Interface B: clean targets, one fixed corruption)**.** Each training sequence is corrupted once, at one level, with the sequences split evenly across the twelve levels, by the same function `_one_view_groups` that builds the structured estimator's data. The pair $(\mathbf{a}, \mathbf{x})$ is kept and never redrawn, and the target $\mathbf{z} = (\mathbf{x} - e^{-t}\mathbf{a})/\sqrt{\Delta_t}$ (or $\mathbf{a}$) is regressed by the same denoising loss on batches drawn from the fixed pairs; the training loop is `_train_select_fixed_corruption`. The checkpoint is selected as in interface A, on the validation bundle against the true score.

**Definition 1.12** (Interface C: one noised view per sequence, no oracle)**.** The network sees only the noised views $(\mathbf{x}^\mu, t_\mu)$ of interface B, without the clean sequences: the same one-view bundle the structured estimator sees. It is trained by implicit score matching (`loss="ism"`), with the divergence computed exactly for the window head and by Hutchinson's estimator for the transformer. In E03 the exact trace of the window head is the $L$-pass branch (`hutchinson=(arch != "window")` with no `trace` override); in E14, whose network arms train exactly as E03's, it is the coloured-probe branch of Proposition 1.7 (`trace_by_arch`), which returns the same number at a fifth of the cost. The checkpoint is selected on the implicit objective $\mathbb{E}[\frac{1}{2}|\widehat{s}|^2 + \operatorname{div}\widehat{s}]$ of a held-out noisy bundle, one view per validation sequence, with the exact trace (`_ism_objective`, `select_on="ism"`); the true score enters only in the final scoring on the test bundle.

In the paper's words (Section 5): (A) is "clean sequences with a fresh corruption at every step"; (B) is "clean sequences with one fixed corruption each"; (C) is "one noised view per sequence and nothing else, which is what EM–BP sees", and the matched comparison adds that "since the true score is not available for selection in that setting, the checkpoint [is] selected on the implicit score-matching objective of a held-out noisy bundle". The sigma-algebras are nested: C sees strictly less than B, which sees strictly less than A (A sees unboundedly many corruptions of each sequence as training proceeds). The audit of 18–19 September 2026 (`docs/SCIENCE_AUDIT.md`) is the reason the paper now names the interface of every number: an earlier draft called the interface-B arm "information-matched", which it is not, because its regression target is the clean sequence.

*Why it matters here.* The paper's headline factor at "matched information" is the ratio of the interface-C network's error to the structured estimator's on the same bundles; it is a statement about what the factorisation and the exact E-step are worth at equal information, and Definition 1.12 is what "equal information" means. Interface B isolates the value of fresh corruptions, and the difference between A and B is that value.

## 1.9 Worked example

**Example 1.13** (The three objectives on a Gaussian)**.** Let $L = 1$, $P_0 = \mathcal{N}(0, 1)$, so that $P_t = \mathcal{N}(0, 1)$ at every $t$ and the score is $S(x) = -x$. Consider the linear family $s_c(x) = -cx$.

- Explicit: $J_{\mathrm{ESM}}(c) = \mathbb{E}(-cx + x)^2 = (1 - c)^2$.

- Implicit: $J_{\mathrm{ISM}}(c) = \mathbb{E}[\frac{1}{2}c^2x^2 + (-c)] = \frac{1}{2}c^2 - c = \frac{1}{2}(c-1)^2 - \frac{1}{2}$. The offset $-\frac{1}{2}$ is $-\frac{1}{2}\mathbb{E}|\nabla \log p|^2$, as Equation (1.2) says, and the minimiser is $c = 1$ in both.
- Denoising: with $x = \alpha a + \sqrt{\Delta_t}z$ and target $-z/\sqrt{\Delta_t}$, $J_{\mathrm{DSM}}(c) = \mathbb{E}(-cx + z/\sqrt{\Delta_t})^2 = c^2\mathbb{E}x^2 - 2c\,\mathbb{E}[xz]/\sqrt{\Delta_t} + \mathbb{E}z^2/\Delta_t = c^2 - 2c + 1/\Delta_t = (c-1)^2 + e^{-2t}/\Delta_t$, since $\mathbb{E}[xz] = \sqrt{\Delta_t}$ and $1/\Delta_t - 1 = e^{-2t}/\Delta_t$. The offset is the irreducible term $(e^{-2t}/\Delta_t^2)\mathrm{MMSE}_t$ of Corollary 1.3 with $\mathrm{MMSE}_t = \mathrm{Var}(a \mid x) = 1 - \alpha^2/(\alpha^2 + \Delta_t) = \Delta_t$. At $t = \log 2$ it equals $(1/4)/(3/4) = \frac{1}{3}$, the SNR.

The `eps` loss for the same family is $\Delta_t J_{\mathrm{DSM}}$ and the `x0` loss is $(\Delta_t^2/\alpha^2)J_{\mathrm{DSM}}$: at $t = \log 2$ these are $\frac{3}{4}J_{\mathrm{DSM}}$ and $\frac{9}{4}J_{\mathrm{DSM}}$, and at level 1 of **??** they are $0.095\,J_{\mathrm{DSM}}$ and $0.010\,J_{\mathrm{DSM}}$.

*Where it is used.* Score matching is the training principle of every network in the paper's Section 5 and Appendix C; the interfaces are defined in Section 5 and used in the baselines and matched-comparison paragraphs; the estimand is Appendix C. Experiments: E03 (screen, confirmatory table, matched arm, length extrapolation), E08 (misspecification network arm), E14 (regime baselines, whose network arms train exactly as E03's: window head with the exact trace by coloured probes, transformer with Hutchinson's estimator), and E05 for the network samplers. Code: `src/markov_score/nn_torch.py` (`TrainConfig`, `train`, `dsm_loss`, `ism_loss`, `divergence`, `ism_objective`, `Denoiser`), `experiments/e03_baselines.py` (`_train_select`, `_one_view_groups`, `_train_select_fixed_corruption`, `_ism_objective`, `_risk`, `_sq`), `experiments/protocol.py` (`estimand`, `bulk_margin`, `net_steps`), `tests/test_nn_torch.py`.

**References.** A. Hyvärinen, 2005, "Estimation of non-normalized statistical models by score matching", Journal of Machine Learning Research 6, 695–709. P. Vincent, 2011, "A connection between score matching and denoising autoencoders", Neural Computation 23, 1661–1674. M. F. Hutchinson, 1990, "A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines", Communications in Statistics: Simulation and Computation 19, 433–450. A. R. Curtis, M. J. D. Powell and J. K. Reid, 1974, "On the estimation of sparse Jacobian matrices", Journal of the Institute of Mathematics and its Applications 13, 117–119. Y. Song, S. Garg, J. Shi and S. Ermon, 2019, "Sliced score matching: a scalable approach to density and score estimation", Uncertainty in Artificial Intelligence. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, 2021, "Score-based generative modeling through stochastic differential equations", International Conference on Learning Representations.